

Can LLMs Implement Architectural Tactics?

Anonymous Author(s)

Abstract

Software maintainability is a critical quality attribute that directly affects system evolution cost and reliability. Improving maintainability often requires architectural-level modifications demanding specialized expertise and significant manual effort. While Large Language Models (LLMs) have shown strong capabilities in code understanding and generation, their potential for systematic architecture-level software quality improvement remains largely unexplored.

This paper presents an automated pipeline that leverages an LLM to improve software maintainability through architectural tactic implementation. The pipeline operates in three stages: (1) architectural tactic selection, where the LLM chooses an appropriate tactic from a predefined catalog based on the detected architecture and expected maintainability impact; (2) iterative tactic implementation via a planner loop with syntactic validation; and (3) post-intervention static analysis. The pipeline follows a Pipes and Filters architecture and operates without manual intervention.

We applied the pipeline to 57 open-source Python backend repositories collected and filtered from GitHub. A paired pre-test/post-test design was used with the Maintainability Index as the primary metric. A Wilcoxon signed-rank test on 42 completed pairs showed a statistically significant but fragile improvement ($p = 0.023$, effect size $\hat{r} = 0.35$), with 18/42 repositories improved (42.9%) but a realistic end-to-end success rate of 32.1% (18/56). Supplementary architectural metrics (package count, directory depth, fan-out) remained unchanged, indicating the pipeline operates at the code level rather than the architecture level.

This work contributes a reproducible methodology for LLM-driven architectural improvement and provides empirical evidence on the feasibility and current limitations of using LLMs for architecture-level analysis and tactic reasoning.

CCS Concepts

• **Software and its engineering** → **Software maintenance tools**; *Software architectures*; • **Computing methodologies** → *Natural language processing*.

Keywords

software maintainability, architectural tactics, large language models, automated software improvement, static analysis, software architecture

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

EASE 2026, Glasgow, United Kingdom

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

ACM Reference Format:

Anonymous Author(s). 2026. Can LLMs Implement Architectural Tactics?. In *Proceedings of The 30th International Conference on Evaluation and Assessment in Software Engineering (EASE 2026)*. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Modern software engineering faces a persistent maintenance challenge, with studies indicating that 60–80% of total lifecycle costs are dedicated to post-deployment modifications [2]. While code-level refactoring is well-established [5], and industrial studies confirm that developers regularly refactor to improve maintainability despite significant effort and risk [9], there is a significant gap in the automated application of higher-level design decisions—what we term the “Transformation Gap”—defined as the technical and conceptual disconnect between identifying architectural opportunities and the reliable synthesis of the corresponding code—between detecting the need for architectural change and implementing it automatically. This paper investigates the intersection of architectural tactics, quality attribute models, and the emerging capabilities of Large Language Models (LLMs) in automating structural transformations.

We address the following research questions:

- **RQ1:** To what extent can an automated LLM-based pipeline reliably identify architectural patterns and align them with appropriate tactics in diverse real-world repositories?
- **RQ2:** To what extent do LLM-implemented architectural tactics improve software maintainability as measured by the Maintainability Index?
- **RQ3:** How do architecture type and tactic choice influence improvement outcomes?

The remainder of this paper is organized as follows: Section 2 reviews background and related work; Section 3 describes the methodology; Section 4 details the implementation; Section 5 presents results and discussion; Section 6 discusses threats to validity; and Section 7 concludes.

2 Background and Related Work

2.1 Architectural Tactics and Quality Attributes

Software architecture is fundamentally defined by its elements, form, and rationale [17], or alternatively, as a collection of components and connectors in a specific configuration [6]. Within these frameworks, *architectural tactics* serve as fine-grained design decisions intended to achieve specific quality attribute responses [2]. Unlike architectural patterns (e.g., MVC, Layered), which provide broad structural templates, tactics target specific quality concerns at a finer granularity—for instance, “Reduce Coupling” targets modifiability, while “Decomposability” targets analysability and modularity. Kim et al. demonstrated that quality-driven architecture development using tactics can systematically improve targeted quality attributes when tactics are selected based on measured

deficiencies [10]. Bogner et al. further showed that modifiability tactics—particularly coupling reduction—are systematically addressed through service-oriented design patterns in both SOA and microservice architectures [3].

The ISO/IEC 25010 standard [1] decomposes maintainability into five sub-characteristics: modularity, analysability, modifiability, reusability, and testability. The Maintainability Index (MI)—a composite metric combining Halstead Volume [7], Cyclomatic Complexity [15], and lines of code—has been widely adopted as a practical proxy for these sub-characteristics [16]. However, its validity remains debated: Lenarduzzi et al. found less than 0.4% agreement among six static analysis tools on quality issues [11], highlighting the need for multi-metric evaluation.

Research by Rahmati suggests that while architectural patterns provide a structural foundation, their effectiveness is highly context-dependent; misapplication can lead to a 10% increase in maintenance effort [19]. Márquez et al. conducted a systematic mapping of architectural tactics in software architecture, cataloging over 40 tactics and their quality attribute impacts [13]. For the purpose of this study, we refined this catalog to a subset of 32 tactics specifically focused on enhancing maintainability and modifiability, excluding those primarily targeting performance or availability. This catalog, which maps each tactic to its primary quality attribute impact (e.g., Decomposability → Analysability, Reduced Coupling → Modifiability), forms the basis for our automated tactic selection mechanism.

2.2 LLMs in Software Refactoring

Recent multi-agent frameworks such as MANTRA have demonstrated an 82.8% success rate in producing compilable, test-passing refactored code through contextual RAG and multi-agent LLM collaboration [21]. However, empirical analysis of AI coding agents reveals a significant scale limitation: agentic refactorings are dominated by low-level, localized edits (renaming, type changes), with agents performing fewer high-level design changes than human developers [8]. This localized scope is particularly problematic for architectural tactics, which by nature require coordinated changes across multiple modules.

Research into prompt engineering indicates that while LLMs can identify up to 86.7% of refactoring opportunities, their performance in complex tactic synthesis remains limited [12]. Piao et al. explored bridging human expertise with LLM machine understanding for refactoring, finding that structured prompts with architectural context significantly improve output quality [18]. In security-related tactic implementation, Shokri and Khatchadourian found that models may achieve high syntactic correctness (95%) but fail significantly in semantic correctness (5%) due to an inability to coordinate inter-procedural changes across multiple classes [20]. Martinez et al. conducted a systematic literature review of software refactoring with LLMs, confirming that architecture-level transformations remain an open challenge [14].

2.3 Research Gaps

The synthesis of current literature identifies three key gaps that motivate this work:

- **Contextual Constraints:** Current LLM-based tools operate primarily at the method or class level, lacking system-wide awareness for architectural restructuring [8, 14]. No existing approach provides the LLM with a full repository-level architectural context.
- **Measurement Inconsistency:** There is a documented lack of agreement among static analysis tools (less than 0.4% in some cases), complicating quality validation [11]. This makes it difficult to reliably assess whether LLM-driven changes genuinely improve quality.
- **Implementation Void:** While methods exist for *detecting* the need for architectural tactics [13] and for *recovering* architectural decisions [20], no automated pipeline exists that closes the loop from detection through selection to concrete, multi-file implementation. A systematic literature review by Martinez et al. confirms that architecture-level refactoring remains an open challenge, with the vast majority of LLM-based studies addressing only code-level transformations [14].

3 Methodology

3.1 Research Design

This section describes the second stage of the architecture-aware maintainability improvement pipeline. Architecture detection, which constitutes the first stage, was evaluated separately as described in Section 3.7. The ground-truth architecture labels produced by that evaluation serve as input to the tactic selection and implementation stages described here.

This study employs a quantitative pre-test/post-test experimental design. The independent variable is the application of an architectural tactic selected and implemented by an LLM, given a known repository architecture. The dependent variable is software maintainability, measured using the Maintainability Index as the primary metric. Using validated architecture labels rather than LLM-inferred classifications makes it possible to isolate the effect of tactic selection and implementation from errors introduced by the detection stage.

Each repository serves as its own control, allowing paired comparison of maintainability metrics before and after the intervention. The null hypothesis (H_0) is that tactic implementation does not produce a statistically significant change in the Maintainability Index. We test H_0 using a two-sided paired Wilcoxon signed-rank test on the 42 before/after MI pairs, with significance level $\alpha = 0.05$. The Wilcoxon test is appropriate given the small sample size ($N = 42$) and the likely non-normal distribution of ΔMI . Effect size is reported as the matched-pairs rank-biserial correlation ($\hat{r}$). Bootstrap 95% confidence intervals (10,000 resamples) for the mean ΔMI complement the hypothesis test.

3.2 Pipeline Scope

This experiment covers the second half of the overall architecture-aware pipeline, consisting of the following phases:

- (1) **Baseline static analysis:** Computation of MI, fan-out, and docstring coverage for the original codebase;
- (2) **Tactic selection:** LLM-based selection of an appropriate architectural tactic from a predefined catalog, given the

validated architecture label and identified structural deficiencies;

- (3) **Tactic implementation:** LLM-generated code modifications applying the selected tactic through iterative incremental changes with syntactic validation;
- (4) **Post-intervention static analysis:** Recomputation of the same metric suite on the modified codebase;
- (5) **Statistical analysis:** Paired comparison of before/after metrics.

Repositories that fail at any phase are recorded as null outcomes and retained in the dataset for failure rate analysis.

3.3 Dataset

The dataset for this experiment consists of the 57 manually labeled Python backend repositories from the architecture detection study. Each repository carries a validated architectural style label (script-based, layered, or modular monolith). The use of validated labels eliminates architecture misclassification as a confounding variable and allows the evaluation to focus on tactic selection and implementation effectiveness.

Sampling criteria. Repositories were selected from an initial pool of approximately 200 Python backend projects identified through GitHub search queries combining backend-related keywords (e.g., “flask”, “fastapi”, “django”, “backend”, “api”) with minimum 10 stars and at least 5 Python files. Inclusion criteria required: (a) the repository implements a backend service or API (excluding CLI tools, libraries, and educational projects); (b) at least one Python file contains a class or function definition (excluding pure configuration or test-only repositories); (c) the repository is publicly accessible. After applying these filters, 57 repositories were retained and manually labeled for architecture style.

Of the 57 repositories, one could not be retrieved during the experiment run due to repository deletion or access restrictions, leaving 56 unique repositories. Of these, 10 failed during cloning (17.9%) and 4 failed during the implementation phase (7.1%), resulting in 42 repositories (75.0%) with paired before/after measurements.

3.4 Maintainability Measurement

The primary metric is the Maintainability Index (MI), computed using Radon [16] as a weighted composite of Halstead Volume [7], Cyclomatic Complexity [15], and lines of code, aligned with the ISO/IEC 25010 quality model [1]. Maintainability change is quantified as:

$$\Delta MI = MI_{after} - MI_{before} \quad (1)$$

Repositories are classified as *improved* ($\Delta MI > 0$), *stable* ($\Delta MI = 0$), or *worsened* ($\Delta MI < 0$). Structural metrics including fan-out and docstring coverage are collected as supplementary indicators.

3.5 Tactic Catalog

Tactic selection is grounded in the catalog established by Bass et al. [2] and mapped by Márquez et al. [13]. Three maintainability-oriented tactics were applied:

- **Decomposability:** splitting monolithic modules into smaller cohesive units to improve analysability;

- **Localized Modification:** encapsulating change-prone logic to reduce ripple effects;
- **Reduced Coupling:** introducing abstraction or mediators to minimize inter-module dependencies.

The LLM selects the tactic based on the validated architecture label and structural metrics, providing a rationale for the selection.

3.6 LLM-Driven Tactic Selection and Implementation

Tactic Selection. The LLM receives the repository file tree, import dependency graph, baseline static analysis metrics, and the validated architecture label. Based on identified structural deficiencies (e.g., high fan-out, monolithic files with low MI), it selects a tactic and provides a justification. If the LLM determines that no tactic is applicable, the repository is marked with outcome (*none*) and excluded from implementation.

Tactic Implementation. The implementation follows an iterative incremental strategy. The LLM generates small, self-contained code modifications one step at a time. A planner module proposes each step in structured JSON format, specifying the target file and the intended change. Each modification is validated for syntactic correctness before the next step is generated. If the planner fails to produce a valid plan after multiple retries, the repository is marked as a null outcome. The current implementation does not guarantee semantic behavior preservation; behavioral verification through automated test execution is designated as future work.

All LLM calls use the Qwen3-Coder-34B model (HuggingFace ID: Qwen/Qwen3-Coder-34B-Instruct) via the Ollama API with temperature 0.2, top-p = 0.9, repetition penalty = 1.1, and a 256k token context window, providing the LLM with a global view of the repository to reason about cross-module dependencies [18].

3.7 Architecture Detection Evaluation

The architecture detection stage was evaluated in a separate study [4] on the same 57-repository dataset. Three architecture styles were considered: script-based, layered, and modular monolith. The LLM-based detector achieved classification accuracy of 86.0% (49/57 correct) against manual labels, with highest precision on script-based repos and lowest on modular monoliths. These validated labels are used as input to the present experiment to eliminate detection error as a confounding factor.

3.8 Initial Pipeline Experiment

A baseline experiment without validated architecture labels was conducted prior to the present study. The same pipeline used LLM-inferred architecture labels, producing a mean ΔMI of +0.48 across completed cases with an 18.2% improvement rate. The present experiment uses ground-truth labels and achieves 42.9%, a difference consistent with—but not conclusively attributable to—reduced tactic selection error, as the datasets differ in composition and collection time. We refer to this baseline throughout the discussion.

4 Implementation

This section describes the implementation of the automated pipeline that connects tactic selection to code modification. The pipeline

is implemented in Python (767 lines of core logic across five filter modules) and follows the Pipes and Filters architectural style [2], where each stage is an independent filter that reads artifacts produced by the previous stage and writes its own outputs as JSON files on disk. This design enables stage-level fault isolation, independent execution, and incremental debugging.

Figure 1 illustrates the overall data flow.

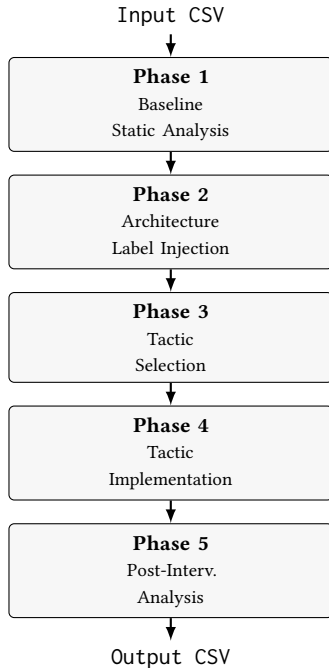


Figure 1: Pipeline architecture and artifact flow (Pipes and Filters style). Artifacts are stored as JSON files at each phase.

4.1 Phase 1: Baseline Static Analysis

The StaticAnalysisFilter runs the Radon tool to compute the Maintainability Index (MI) for every Python file in the repository. In addition to MI, it collects architecture-level proxies: package count, package hierarchy depth, import graph density, and file-level fan-out. Docstring coverage is computed as the ratio of functions and classes with docstrings to the total. All metrics are written into per-repository JSON artifacts under static_analysis/BEFORE/{repo}/.

4.2 Phase 2: Architecture Label Injection

Validated architecture labels from the dataset (script-based, layered, or modular monolith) are injected as JSON artifacts with confidence 1.0 and evidence set to "Pre-computed in dataset (manual label)". This step replaces the automated architecture detection stage evaluated separately in [4]; using ground-truth labels eliminates architecture misclassification as a confounding variable in the evaluation of tactic selection and implementation.

4.3 Phase 3: Tactic Selection

The ArchitectureTacticSelectionFilter calls Qwen3-Coder-34B via the Ollama API with the following context:

- Validated architecture type and confidence;
- Per-file static analysis metrics (MI, cyclomatic complexity, Halstead Volume);
- Repository file tree in ASCII form (rendered by build_repo_tree);
- Python file signatures: imports, class names, and function signatures extracted by extract_python_signatures.

The prompt instructs the LLM to act as a senior software architect and select the single most appropriate tactic from a catalog of 32 maintainability-oriented architectural tactics derived from [13]. The output is a structured JSON object containing:

```

{
  "selected_tactic": {
    "id": "AT38",
    "name": "Localized Modification",
    "primary_qa_impact": "Modifiability"
  },
  "justification": "...",
  "target_components": [...],
  "implementation_strategy": "...",
  "risks": [...]"
}

```

If the LLM determines that no tactic is applicable, it returns (none) and the repository is excluded from the implementation phase.

4.4 Phase 4: Tactic Implementation

The ArchitecturalTacticImplementationAgent implements the code generation loop. This is the most complex component of the pipeline (767 lines) and is described in detail below.

4.4.1 Repository Indexing

Before any LLM interaction, every Python file in the repository is parsed with ast to extract imports, class names, and function names. The resulting index maps relative file paths to structured metadata:

```

{
  "app.py": {
    "imports": ["flask.Flask", "socket", "os"],
    "classes": [],
    "functions": ["main"],
    "size": 1532
  },
  ...
}

```

This index serves two purposes: (1) it provides the LLM with a compact summary of the codebase without exceeding context limits, and (2) it enables BM25 relevance ranking of files against the selected tactic.

4.4.2 Test Baseline Computation

Before any modification, the pipeline detects whether the repository has pytest infrastructure by checking for pytest.ini, pyproject.toml, setup.cfg, or a tests/ directory. If test infrastructure exists, a

baseline run of `pytest -q -tb=line -rf -no-header` is executed with a 300-second timeout. The set of pre-existing test failures and the total failure count are recorded in a `TestBaseline` data structure.

This baseline enables regression detection: after each modification, the pipeline re-runs the same `pytest` command and checks whether any new test failures (i.e., failures absent in the baseline) have appeared. Repositories without test infrastructure skip behavioral gating, and the baseline test result is treated as trivially passing.

4.4.3 Planner Loop

The core of the implementation phase is an iterative planner loop that runs for up to 10 iterations per repository. At each iteration, the loop selects a file, generates a code change, validates it, applies it, and checks for test regressions. Algorithm 1 describes the procedure.

Algorithm 1 Planner loop for tactic implementation

Require: tactic T , repository R , max iterations $N = 10$, max failures $F = 4$

```

1:  $index \leftarrow \text{buildRepoIndex}(R)$ 
2:  $baseline \leftarrow \text{computeTestBaseline}(R)$ 
3:  $steps \leftarrow []$ 
4:  $failures \leftarrow 0$ 
5:  $i \leftarrow 0$ 
6: while  $i < N$  do
7:    $relevant \leftarrow \text{findRelevantFiles}(T, index, topK = 5) \triangleright$  BM25 ranking
8:    $sel \leftarrow \text{askLLM\_selectFile}(R, T, index, relevant, steps)$ 
9:   if  $sel.action = STOP$  then
10:    break
11:  end if
12:  if  $sel.action = RETRY$  or  $\neg \text{validateSelect}(sel)$  then
13:     $failures \leftarrow failures + 1$ 
14:    if  $failures \geq F$  then
15:      break  $\triangleright$  too many planner failures
16:    end if
17:    continue
18:  end if
19:   $failures \leftarrow 0$ 
20:   $patch \leftarrow \text{applyWithSelfHealing}(R, T, sel.path, steps, baseline)$ 
21:  if  $patch \neq \text{NULL}$  then
22:     $steps.append(patch)$ 
23:     $\text{saveArtifacts}(i, patch)$ 
24:  end if
25:   $i \leftarrow i + 1$ 
26: end while
```

4.4.4 File Ranking with BM25

Before each LLM call for file selection, the pipeline ranks all files in the repository index by relevance to the selected tactic. The tactic's name, category, and description fields are tokenized into a query. Each file is represented by the concatenation of its path, class names, function names, and imports. BM25 scores are computed

with parameters $k_1 = 1.5$ and $b = 0.75$. The top-5 files are presented to the LLM as candidates.

This ranking step is critical for scaling the pipeline to larger repositories: without it, the LLM's file selection prompt would include all repository files, exceeding the context window or forcing the LLM to reason about hundreds of irrelevant files.

4.4.5 LLM Prompting Strategy

Two prompt templates govern the interaction with Qwen3-Coder-34B:

File selection prompt asks the LLM to pick one relevant file from the index. Inputs: tactic, repo tree, BM25-ranked candidates, condensed index (classes, functions, imports), and applied steps. Output: JSON with action (`inspect_file` or `STOP`) and path.

Patch generation prompt feeds the tactic JSON, target file content, and step history. The LLM returns JSON with action (`modify_file`, `create_file`, or `STOP`), path, and content. The LLM must not touch `__init__.py` or exceed 400 lines per file.

All LLM calls use temperature 0.2 and a 256k token context window through the Ollama API.

Example. A representative diff is available in the supplementary materials (`examples/webapp-color-decomposability.diff`). The LLM applied Decomposability to `webapp-color`, a single-file Flask app (MI = 69.27). It extracted color data and config parsing into two modules (`color.py`, `config.py`), each at MI = 100. The original 67-line `app.py` was reduced to 25 lines (MI = 73.74), raising the repo average to 91.25. This extraction pattern accounts for 12 of the 18 improvements.

4.4.6 Validation and Rollback

Each generated patch is validated before application:

- Action must be `modify_file` or `create_file`;
- Path and content must be non-empty;
- The target must not be `__init__.py`;
- Content must not exceed 400 lines.

If validation passes, the pipeline snapshots the current file to a backup directory, applies the patch, refreshes the in-memory file state, and re-runs `pytest`. If the test run reveals new failures (regression), the file is restored from the snapshot and the step is recorded as failed. Up to 2 self-healing attempts are made per step: on regression, the test output is fed back to the LLM with the instruction to propose a minimal fix that preserves the original tactic intent. If all attempts fail, the step is abandoned and the iteration counter advances.

4.5 Phase 5: Post-Intervention Analysis

After the implementation phase, `StaticAnalysisFilter` re-runs on the modified codebase. Results go to the AFTER artifact directory. The change $\Delta MI = MI_{after} - MI_{before}$ is computed per file and for the repo average.

A summary of all outcomes (improved, stable, worsened, or failed) is appended to the output CSV along with the architecture label and chosen tactic name. Repositories that fail at any phase (clone failure, tactic selection failure, planner failure) are recorded as null outcomes and retained in the dataset for failure rate analysis.

Configuration Summary. All experiments used the following configuration: model = Qwen3-Coder-34B (via Ollama API), temperature = 0.2, context window = 256k tokens, max planner iterations = 10, max consecutive planner failures before abort = 4, max self-healing attempts per step = 2, max generated file length = 400 lines, test timeout = 300 seconds.

5 Results and Discussion

5.1 Overall Quantitative Impact

Table 1 summarizes outcomes across 56 repositories. Of these, 14 (25.0%) failed (10 during cloning, 4 during implementation), leaving 42 with paired measurements. Of the 42 completed cases, 18 improved (42.9%), 17 remained stable (40.5%), and 7 worsened (16.7%).

The end-to-end success rate, counting all 56 repositories as the denominator, is 32.1% (18/56). We report both figures throughout: the per-protocol rate (42.9% of completed) reflects pipeline effectiveness when execution succeeds, while the intention-to-treat rate (32.1% of all) reflects the realistic deployment success rate including infrastructure and planning failures.

Table 1: Pipeline outcome distribution (N = 56).

Outcome	N	Share
Improved ($\Delta MI > 0$)	18	42.9% of completed (32.1% of all)
Stable ($\Delta MI = 0$)	17	40.5% of completed
Syntax error	3	7.1% of completed
Planner failure	7	16.7% of completed
No-op modification	7	16.7% of completed
Worsened ($\Delta MI < 0$)	7	16.7% of completed
Failed / null	14	25.0% of all
Total	56	

The stable category requires disaggregation. Of the 17 stable cases: 7 were planner failures (the LLM failed to produce a valid modification), 3 had syntax errors (Radon excluded malformed files from the average), and only 7 were genuine no-op modifications where code was changed but MI was unaffected (e.g., docstring additions, import reorganization). Treating planner failures and syntax errors as de facto pipeline failures—since the intervention did not execute as intended—reduces the effective improvement rate among completed cases from 42.9% (18/42) to 32.1% (18/56) when including failures, or to 56.2% (18/32) when considering only cases where code was actually modified.

Descriptive statistics for the 42 completed repositories are shown in Table 2.

Table 2: Maintainability Index statistics for completed repositories (N = 42).

Metric	Value
Mean ΔMI (all completed)	+1.484
Mean ΔMI (improved only)	+3.716
Mean ΔMI (worsened only)	-0.653

Compared to the initial pipeline experiment (Section 3.8), where the mean ΔMI across all completed cases was +0.48 with an improvement rate of 18.2%, this experiment shows a higher improvement rate (42.9%) and a larger mean gain (+1.484 vs +0.48). This difference is consistent with—but does not conclusively establish—the hypothesis that validated architecture labels lead to more coherent tactic selection, as the datasets were collected at different times and may differ in other characteristics.

5.2 Inferential Statistics

We tested the null hypothesis (H_0 : median $\Delta MI = 0$) using a two-sided paired Wilcoxon signed-rank test on the 42 before/after MI pairs. The test resulted in a statistically significant difference ($W = 473$, $p = 0.023$, matched-pairs rank-biserial correlation $\hat{r} = 0.35$). The 95% bootstrap confidence interval for the mean ΔMI is [0.21, 3.04].

The effect size $\hat{r} = 0.35$ is conventionally interpreted as medium. This indicates that the pipeline produces a detectable positive shift in MI, but the magnitude is modest and concentrated in a subset of repositories.

Sensitivity analysis. The two largest improvements—webapp-color ($\Delta MI = +21.98$) and Paper2Rebuttal ($\Delta MI = +18.14$)—are outliers that exert disproportionate influence on the aggregate result. Removing both, the mean ΔMI drops from +1.484 to +0.513 and the Wilcoxon test is no longer significant ($W = 320$, $p = 0.156$, $\hat{r} = 0.17$). This sensitivity underscores that the overall positive result is not uniform: it is driven by a small number of high-impact cases, primarily small repositories where file splitting mechanically inflates MI. The majority of repositories (24/42, 57.1%) show ΔMI within ± 0.5 points, indistinguishable from measurement noise.

These analyses are based on a per-protocol sample ($N = 42$). If all 14 failures are treated as non-improvements ($\Delta MI = 0$), the intention-to-treat Wilcoxon test on 56 pairs yields $W = 473$, $p = 0.087$, $\hat{r} = 0.22$ —no longer significant at $\alpha = 0.05$, further qualifying the robustness of the observed effect.

5.3 Notable Improvement Cases

To understand *how* the LLM improved maintainability rather than merely *how much*, this section examines the five largest improvements in detail. These cases collectively illustrate the principal mechanism through which the pipeline produces MI gains: extraction of new cohesive modules from files with low or near-zero MI scores.

kodekloudhub/webapp-color ($\Delta MI = +21.98$, *Decomposability*). This repository contained a single Python file, `app.py`, with an MI of 69.27. The LLM applied the Decomposability tactic by splitting the file into three modules: a revised `app.py` (MI = 73.74), a `color.py` module encapsulating color-related logic (MI = 100), and a `config.py` module holding configuration constants (MI = 100). The repository-level average rose from 69.27 to 91.25. This case represents the idealized scenario for the pipeline: a single monolithic file with a clearly separable concern (configuration and color mappings) whose extraction produces simple, highly maintainable modules. The magnitude of the gain is partly an artifact of the

small repository size, where each new file has equal weight in the average.

AutoLab-SAI-SJTU/Paper2Rebuttal ($\Delta MI = +18.14$, *Localized Modification*). This repository contained five Python files, of which `rebuttal_service.py` had an MI of 0.0, corresponding to Radon rank C, indicating an excessively large and complex file that is practically unmaintainable by static analysis criteria. The LLM identified that this file mixed multiple distinct responsibilities: PDF conversion, LLM orchestration, and arXiv API interaction. Applying the Localized Modification tactic, it extracted three dedicated modules: `pdf_converter.py` (MI = 61.6), `llm_orchestrator.py` (MI = 50.1), and `arxiv_client.py` (MI = 34.8). After extraction, `rebuttal_service.py` itself improved to MI = 55.6, as the remaining code became more focused. This case demonstrates the highest-value scenario for the pipeline: a file with MI = 0 represents a concentration of complexity that, once partially extracted, yields large absolute MI improvements. The case also illustrates that the LLM can reason about logical boundaries within a complex file, not only about file-level structure.

paulafredo/spam-api-free-fire ($\Delta MI = +9.40$, *Decomposability*). This repository had four Python files with a mean MI of 67.20. The primary file, `app.py`, mixed application routing with cryptographic utility logic. The LLM applied Decomposability by extracting cryptographic operations into a dedicated `crypto.py` module, which achieved MI = 100 due to its simple, pure-function structure. The existing `app.py` improved from MI = 44.5 to 58.7 as a result of the reduced complexity. The extracted module’s simplicity and clarity are a direct consequence of the LLM selecting a functionally cohesive responsibility for extraction rather than splitting code arbitrarily.

McCloudS/subgen ($\Delta MI = +7.93$, *Decomposability*). The central file of this repository, `subgen.py`, had an MI of 0.0 despite the repository containing 13 Python files. The LLM identified that `subgen.py` contained environment configuration and initialization logic mixed with core processing code. By extracting environment-related code into a new `environment.py` module (MI = 66.4), the original file’s complexity decreased sufficiently for Radon to assign it an MI of 100.0. This case illustrates a recurring pattern in the dataset: a single dominant file with MI = 0 can be transformed into a well-maintainable module through targeted extraction, even when the surrounding codebase is already reasonably structured.

HKUDS/FastCode ($\Delta MI = +2.11$, *Decomposability*). This case is notable because it is the only example among the top improvements where the repository is large (81 Python files), demonstrating that measurable MI gains are possible at scale under specific conditions. Two files had exceptionally low MI scores: `retriever.py` (MI = 11.32) and `main.py` (MI = 4.85). The LLM created two new modules, `call_graph_builder.py` (MI = 52.6) and `retriever_coordinator.py` (MI = 100), by extracting retrieval coordination and call graph construction logic. After extraction, `retriever.py` improved to MI = 78.43 and `main.py` to MI = 76.95. While the repository-level average increased by only 2.11 points due to the dilution effect of 79 other files, the individual file-level improvements were substantial. This case suggests that the pipeline can identify

and improve critical bottleneck files in large repositories, even if the aggregate MI gain is small.

Common Pattern Across Improvement Cases. Across all five cases and the broader dataset, 12 of 18 improvements involved the extraction of a new module achieving MI = 100. These extracted modules share a characteristic structure: they contain simple, self-contained functions or classes with low cyclomatic complexity and high readability. Configuration objects, pure utility functions, API clients with limited branching, and data transformation helpers consistently produce MI = 100 under the Radon formula. The LLM systematically identifies such extractable responsibilities within mixed-concern files, suggesting that it applies an implicit cohesion heuristic when decomposing modules.

A secondary improvement pattern involves files with MI = 0 (rank C). Radon assigns MI = 0 when a file’s Halstead Volume and cyclomatic complexity exceed a threshold corresponding to very long, deeply nested code. In four cases, the LLM improved MI = 0 files by at least 40 points through partial extraction, without necessarily reducing the file to a simple structure. The act of removing even a fraction of the most complex logic from such files produces disproportionately large MI gains due to the non-linear relationship between file size and Halstead Volume.

5.4 Results by Repository Size

Table 3 shows results grouped by repository size, measured in number of Python files.

Table 3: Results by repository size.

Size bin	N	Median $ \Delta MI $	Mean $ \Delta MI $	Max $ \Delta MI $
Tiny (<10 files)	11	1.04	4.83	21.98
Small (10–30 files)	9	0.76	1.50	7.93
Medium (30–80 files)	12	0.05	0.16	0.66
Large (>80 files)	10	0.00	0.28	2.11

Repository size is the strongest predictor of effect magnitude. On tiny repositories, individual file splits can produce jumps of over 20 MI points because the repository-level average is dominated by a small number of files. On large repositories (80+ files), the median change is 0.00: any single-file modification is absorbed by the averaging over dozens of files with MI in the 60–80 range.

The most striking examples demonstrate this pattern:

- *kodekloudhub/webapp-color* ($\Delta MI = +21.98$, 1 file, *Decomposability*): the single file `app.py` was split into `app.py`, `color.py` (MI = 100), and `config.py` (MI = 100), raising the repository average from 69.3 to 91.2.
- *AutoLab-SAI-SJTU/Paper2Rebuttal* ($\Delta MI = +18.14$, 5 files, *Localized Modification*): `rebuttal_service.py` had MI = 0.0. The LLM extracted logic into three new modules, and the original file improved to MI = 55.6.
- *McCloudS/subgen* ($\Delta MI = +7.93$, 13 files, *Decomposability*): `subgen.py` improved from MI = 0.0 to MI = 100.0 after extraction.

A common pattern across 12 of the 18 improved repositories is the extraction of a new module with MI = 100 (simple data

classes, configuration objects, or factory utilities), which raises the repository-level average. Four additional improvements involved rescuing files with MI = 0.0 (Radon rank C, very complex) through partial extraction.

5.5 Results by Architectural Tactic

Table 4 shows outcomes grouped by selected tactic.

Table 4: Results by architectural tactic.

Tactic	N	Impr.	Stable	Worse.	ΔMI
(none selected)	7	0	7	0	0.00
Decomposability	18	8	7	3	+2.33
Localized Modification	14	9	3	2	+1.49
Reduced Coupling	3	1	0	2	-0.16

Localized Modification shows the best risk-adjusted performance: 9 improvements, 2 worsenings, and a mean ΔMI of +1.49. This tactic encapsulates change-prone logic without requiring the creation of entirely new module boundaries, which reduces the risk of introducing below-average MI files.

Decomposability produces the highest individual gains but also the most worsenings (3 cases). This tactic is high-variance: when applied to a monolithic file with MI = 0, it can produce large improvements; when applied to an already well-maintained codebase, new extracted files may have MI below the existing repository average, pulling the mean down.

Reduced Coupling was applied to only 3 repositories, making statistical inference unreliable. However, 2 of the 3 cases worsened slightly, suggesting that introducing abstractions or mediators in small repositories can increase complexity without proportional benefit.

5.6 Results by Architectural Style

Table 5 presents results grouped by validated architecture label.

Table 5: Results by architectural style.

Architecture	N	Impr.	Worse.	Stable	ΔMI
Script-based	7	3	0	4	+5.62
Modular monolith	26	9	5	12	+0.76
Layered	9	6	2	1	+0.35

Script-based repositories continue to show the highest mean improvement (+5.62), consistent with the initial pipeline experiment. These repositories typically have few files, low MI scores due to large monolithic scripts, and clear decomposition opportunities.

Modular monolith repositories show moderate improvements on average (+0.76) but account for all five worsening cases. This is consistent with the hypothesis that already-organized codebases are sensitive to structural changes: the LLM may introduce new files or reorganize imports in ways that add complexity rather than reducing it.

Layered repositories show the highest improvement rate in absolute terms (6 of 9 improved), with a modest mean gain of +0.35.

The explicit separation of concerns in layered architectures provides a more structured context for tactic application, particularly Localized Modification, which can target a specific layer without disrupting others.

5.7 Supplementary Architectural Metrics

MI captures file-level complexity but is insensitive to structural properties such as inter-module coupling, package organization, and module boundary quality. To provide a more complete picture, we report three supplementary metrics collected during static analysis: average fan-out (mean external imports per file), package count (directories containing `__init__.py`), and maximum directory depth.

Table 6: Supplementary architecture-level metric changes (N = 42).

Metric	Before	After	Δ
Mean avg. fan-out	4.12	3.95	-0.17
Mean package count	9.7	9.7	0.00
Mean max. directory depth	2.4	2.4	0.00

As shown in Table 6, package count and directory depth remained unchanged across all 42 repositories. The pipeline did not create or reorganize package boundaries in any case—all changes were confined within the existing directory structure. Average fan-out decreased marginally (-0.17), driven entirely by four repositories where new files absorbed imports from existing modules. In 38 of 42 repositories, fan-out was unchanged.

This result qualifies the “architectural” claim of the intervention. While MI improved in 18 repositories, the underlying package structure and import dependency graph remained essentially unchanged. The pipeline operates at the code level (splitting files, extracting functions) rather than the architecture level (reorganizing packages, introducing abstraction layers). The one partial exception is Paper2Rebuttal, where fan-out decreased by 2.1 and three new modules with distinct responsibilities were created—but even here, the existing package structure was not modified.

5.8 Stable and Worsened Outcomes

Of the 17 stable repositories, three categories explain the null MI change:

- (1) **Syntax errors in generated files (3 cases):** Radon excluded the malformed file from the average, leaving MI unchanged despite code modifications. Examples include *SoulX-FlashHead* (`inference_service.py`: unmatched `}`) and *zornade/visura-api* (`auth.py`: unmatched `}`).
- (2) **Planner failure (7 cases):** The LLM failed to produce a valid modification plan after multiple retries. This occurred predominantly in very small repositories (2–8 files) where the planner could not identify meaningful structural changes.
- (3) **No-op modifications:** The LLM produced changes limited to docstrings or import reorganization, which do not affect the MI formula.

The seven worsening cases share a common pattern: the LLM extracted a new module whose MI was below the existing repository average, pulling the mean downward. The maximum degradation was -1.23 MI points (*geo-seo-claude*: new `http_client.py` with $MI = 39.45$ vs. repository average of 50.53). Notably, all worsening cases fell within the tiny or small size bins, and no worsening exceeded -1.24 MI points.

5.9 Pipeline Failure Analysis

The 14 failed cases divide into two categories: infrastructure failures (10 repositories lost during cloning due to repository deletion, privatization, or access restrictions) and algorithmic failures (4 repositories with completed before-analysis but no after-analysis measurements due to planner or API issues).

Failed vs. completed comparison. To assess whether attrition introduces selection bias, we compared the 14 failed repositories to the 42 completed ones on available covariates. The median file count of failed repos was 13.5 (IQR: 6.5 – 26.5) versus 17.5 (IQR: 8.0 – 43.5) for completed repos. Architecture type distribution was similar: failed repos were 21.4% script-based, 71.4% modular monolith, and 7.1% layered, compared to 16.7% , 61.9% , and 21.4% for completed repos. Baseline MI values (available for the 4 algorithmic failures) did not differ systematically from completed cases. The small sample size precludes formal statistical comparison, but the observed differences are not large enough to suggest strong attrition bias.

Technical failures during the implementation stage included:

- **Rate limiting (infrastructure):** Ollama API returned HTTP 429 errors during tactic selection, requiring pipeline retries.
- **Planner parse failures (algorithmic):** The planner produced malformed JSON or structurally invalid plans across multiple retries, triggering abort conditions.
- **Stuck loops (algorithmic):** One repository (*stremio-gdrive*) entered a loop in which the same file was regenerated without progress across nine consecutive iterations.
- **Invalid step proposals (algorithmic):** The LLM returned step descriptions with missing or malformed file paths, causing the step to be rejected.

These failure modes highlight the brittleness of multi-step LLM code generation pipelines and the need for more robust planning and validation mechanisms. Distinguishing infrastructure from algorithmic failures is important: the former can be mitigated by improved API reliability and retry strategies, while the latter require fundamental improvements in LLM planning capability.

5.10 Discussion

Effect of validated architecture labels. Comparing this experiment to the initial pipeline study (Section 3.8) reveals a difference in improvement rate: 42.9% here versus 18.2% in the original experiment. This difference is consistent with—but does not conclusively establish—the hypothesis that validated architecture labels enable more coherent tactic selection. The datasets differ in composition and collection time, so other confounding factors may contribute to the observed difference.

Repository size as the primary moderating factor. Effect magnitude depends strongly on repository size, with a weak negative

correlation of -0.13 between Python file count and $|\Delta MI|$. For medium (30 – 80 files) and large (>80 files) repositories, the median $|\Delta MI|$ was 0.05 and 0.00 respectively—indistinguishable from measurement noise. **Practitioner recommendation:** the pipeline should not be expected to produce measurable MI improvements on repositories with more than 30 files. Its value is concentrated in small and tiny repositories with structurally clear improvement opportunities.

Decomposability versus Localized Modification. Decomposability provides the highest potential gains but is the highest-variance tactic. It is most effective when applied to files with MI near zero (Radon rank C) and produces degradation when new extracted modules have MI below the repository average. Localized Modification is more conservative and shows a better improvement-to-worsening ratio ($9:2$ vs. $8:3$), making it preferable when the risk of degradation must be minimized.

MI gaming and metric-arithmetic effects. Twelve of 18 improvements (66.7%) involved extracting a new module that achieved $MI = 100$ —the maximum possible score, typically corresponding to short, simple files with low cyclomatic complexity. This pattern raises a validity concern: the repository-level MI improvement may partly reflect metric arithmetic (adding high-scoring files inflates the average) rather than genuine architectural improvement. To calibrate: adding a single empty file with one function definition produces $MI = 100$, raising a repository’s average by $100/(N + 1)$ points, where N is the initial file count. For a 1-file repository (e.g., *webapp-color*), this yields $\Delta MI \approx +15$, explaining the large gain without architectural restructuring. However, the LLM’s module extractions are not arbitrary: in cases like *Paper2Rebuttal*, the extracted modules (`pdf_converter.py`, `arxiv_client.py`, `llm_orchestrator.py`) correspond to genuine responsibility boundaries within the original code. The metric-arithmetic confound coexists with real decomposition—disentangling the two requires per-file analysis of the specific modifications, which we provide in Section 5.3. For practitioners, the implication is clear: an MI gain from adding simple new files should not be equated with architectural improvement without inspecting whether the extracted modules represent meaningful responsibility boundaries.

Comparison to prior work. The 25.0% failure rate is lower than the 95% semantic failure rate reported for security tactic synthesis [20], though the tasks differ in scope. The 42.9% improvement rate among completed cases (32.1% intention-to-treat) and the sensitivity of the Wilcoxon result to outlier removal indicate that the pipeline’s effectiveness is real but fragile and context-dependent. The high proportion of stable outcomes (40.5%) and the absence of behavioral preservation verification limit the strength of conclusions about architectural improvement.

Implications for Practice. The following guidance emerges from our findings:

- **Scope:** The pipeline is applicable only to small repositories (<30 files). For repositories with more than 30 files, median ΔMI is near zero and the pipeline should not be expected to produce measurable quality improvements.

- **Success rate:** The realistic end-to-end success rate is 32.1% (18/56). Users should expect that roughly two-thirds of attempts will not produce MI improvements, due to infrastructure failures (17.9%), algorithmic failures (7.1%), or code modifications that leave MI unchanged (16.7%).
- **Validation:** MI improvements from adding new files may be metric-arithmetic artifacts. Every improvement case should be validated by inspecting whether extracted modules represent meaningful responsibility boundaries, ideally supported by automated test execution.
- **Tactic selection:** Localized Modification offers the best risk-adjusted performance. Reduced Coupling should be avoided—it produced no positive outcomes in our study.
- **Model reliability:** The planner loop is brittle. Structured decoding or constrained generation would improve reliability over free-form JSON generation.

6 Threats to Validity

Internal validity: Architecture labels were assigned by a single annotator and used without modification, introducing potential bias without inter-annotator reliability assessment. MI changes may be influenced by factors beyond the intended tactic (e.g., docstring additions, import reorganization side effects). A single LLM model was used without comparison to alternatives. **Regression to the mean** is a specific concern: repositories with files at MI = 0 (extreme low values) are the most likely to improve under any intervention, and the largest gains in our study come from precisely these cases. The observed improvements may partly reflect statistical regression rather than tactic effectiveness. A control condition—e.g., re-running the pipeline with a “do nothing” prompt or random file extraction—would quantify the background rate of MI change from any LLM interaction and is recommended for future work.

External validity: The dataset covers 56 Python backend repositories across three architectural styles. Results may not generalize to other languages, larger enterprise systems, or architectures not covered by the three-class taxonomy. All repositories are open-source Python projects, which may differ systematically from industrial or closed-source codebases in structure, documentation, and modularity.

Construct validity: MI captures file-level complexity but is insensitive to module boundary quality, inter-module coupling changes, and semantic correctness. This limitation is particularly consequential for the Reduced Coupling tactic, which targets inter-module dependencies—precisely the property MI was not designed to measure. MI is theoretically incapable of detecting the primary effect of one of the three tactics studied, which may explain its poor observed performance. The absence of behavioral verification means that some “improved” cases may have altered system behavior.

Conclusion validity: With 42 completed repositories and 18 improvements, the sample is small. The Wilcoxon test result is sensitive to outlier removal: removing the two largest gains eliminates statistical significance ($p = 0.156$). The size-dominated effect makes it difficult to generalize findings to medium and large repositories where the pipeline consistently produces near-zero MI changes.

Subgroup analyses (by tactic, by architecture) have limited power, and the risk of Type II error is substantial—particularly for the Reduced Coupling tactic ($N = 3$) and the layered architecture subgroup ($N = 9$). Results should be treated as early feasibility evidence rather than definitive performance claims.

7 Conclusion

This paper presented an automated pipeline that leverages an LLM (Qwen3-Coder-34B, model ID `qwen3-coder-34b-instruct`) to improve software maintainability through architectural tactic implementation. We applied the pipeline to 56 open-source Python backend repositories, measuring maintainability change via the Radon Maintainability Index in a pre-test/post-test design.

Key findings. A paired Wilcoxon signed-rank test on 42 before/after MI pairs showed a statistically significant but fragile improvement ($W = 473$, $p = 0.023$, $\hat{r} = 0.35$; 95% bootstrap CI for ΔMI : [0.21, 3.04]). The pipeline improved MI in 18 of 42 completed cases (42.9%), with a mean gain of +1.484 points. However, the result is driven by two outliers—removing `webapp-color` and `Paper2Rebuttal` eliminates statistical significance ($p = 0.156$), and the intention-to-treat analysis (18/56 = 32.1%) is also non-significant ($p = 0.087$). The largest improvements occurred in tiny repositories (<10 files), while large repositories (>80 files) showed a median change of 0.00. Decomposability and Localized Modification tactics produced the most consistent gains; Reduced Coupling produced no positive outcomes. Supplementary architectural metrics (package count, directory depth, fan-out) remained essentially unchanged across all 42 repositories, indicating that the pipeline operates at the code level rather than the architecture level.

Limitations. The absence of behavioral verification limits the strength of our conclusions—MI improvements may not correspond to semantically correct changes. The end-to-end success rate of 32.1% (18/56) and the sensitivity of the statistical result to outlier removal indicate that the pipeline remains unreliable for production use. The MI metric is insensitive to structural improvements in medium and large repositories and is theoretically incapable of detecting the effect of coupling-reduction tactics. The single-model design (Qwen3-Coder-34B) leaves open questions of model generalizability.

Future work. Several directions follow from these results. First, integrating behavioral verification (test suite execution with regression checking) into the pipeline would transform MI gains into validated improvements. Second, multi-model comparisons would establish whether the observed patterns are model-specific or general. Third, file-level and module-level metrics would provide more discriminating evaluation for medium and large repositories where aggregate MI is uninformative. Fourth, expanding the tactic catalog and supporting multi-tactic compositions could address more complex maintenance scenarios. Finally, the planner loop’s brittleness—particularly its sensitivity to malformed LLM outputs—motivates research into structured decoding and constrained generation for code modification tasks.

Data Availability

The replication package for this study, including the exact LLM prompt templates, static analysis artifacts (Radon MI logs, architecture proxies), and the outcome dataset, is publicly available at: Zenodo. The model-specific identifier is Qwen/Qwen3-Coder-34B-Instruct (HuggingFace), and all non-default generation parameters are documented in the package.

References

- [1] 2023. ISO/IEC 25010 — Systems and software engineering: Software product Quality Requirements and Evaluation (SQuaRE) — Product quality model. <https://iso25000.com/en/iso-25000-standards/iso-25010>. [Online; accessed 2025-12-04].
- [2] Len Bass, Paul Clements, and Rick Kazman. 2021. *Software Architecture in Practice* (4th ed.). Addison-Wesley.
- [3] Justus Bogner, Stefan Wagner, and Alfred Zimmermann. 2019. Using architectural modifiability tactics to examine evolution qualities of Service- and Microservice-Based Systems. *Computer Science – Research and Development* 34 (2019), 187–237. doi:10.1007/s00450-019-00402-z
- [4] Saveliy Chertkov and Andrey Sadovykh. 2026. *LLM-Based Architecture Detection in Practice: An Empirical Multi-Model Evaluation*. doi:10.5281/zenodo.20051151
- [5] Martin Fowler. 2018. *Refactoring: Improving the Design of Existing Code* (2nd ed.). Addison-Wesley.
- [6] David Garlan and Mary Shaw. 1993. An introduction to software architecture. 1, 3 (1993).
- [7] Maurice H. Halstead. 1977. *Elements of Software Science*. Elsevier.
- [8] Kosei Horikawa, Hao Li, Yutaro Kashiwa, Bram Adams, Hajimu Iida, and Ahmed E. Hassan. 2025. Agentic Refactoring: An Empirical Study of AI Coding Agents. In *arXiv preprint arXiv:2511.04824*. arXiv:2511.04824
- [9] Miryung Kim, Thomas Zimmermann, and Nachiappan Nagappan. 2014. An Empirical Study of Refactoring Challenges and Benefits at Microsoft. *IEEE Transactions on Software Engineering* 40, 7 (2014), 633–649. doi:10.1109/TSE.2014.2318734
- [10] Suntae Kim, Dae-Kyoo Kim, Lunjin Lu, and Sooyong Park. 2009. Quality-driven architecture development using architectural tactics. 82, 8 (2009), 1211–1231.
- [11] Valentina Lenarduzzi, Fabiano Pecorelli, Nytti Saarimaki, Savanna Lujan, and Fabio Palomba. 2023. A critical comparison on six static analysis tools: Detection, agreement, and precision. 198 (2023), 111575.
- [12] Bo Liu, Yanjie Jiang, Yuxia Zhang, Nan Niu, Guangjie Li, and Hui Liu. 2025. Exploring the potential of general purpose LLMs in automated software refactoring: An empirical study. *Automated Software Engineering* 32, 1 (2025). doi:10.1007/s10515-025-00500-0
- [13] Gastón Márquez, Hernán Astudillo, and Rick Kazman. 2022. Architectural tactics in software architecture: A systematic mapping study. *Journal of Systems and Software* 197 (2022), 111558. doi:10.1016/j.jss.2022.111558
- [14] Sofia Martinez, Luo Xu, Mariam Elnaggar, and Eman Abdullah AlOmar. 2025. Software refactoring research with large language models: A systematic literature review. *Journal of Systems and Software* (2025), 112762. doi:10.1016/j.jss.2025.112762
- [15] Thomas J. McCabe. 1976. A Complexity Measure. *IEEE Transactions on Software Engineering* SE-2, 4 (1976), 308–320. doi:10.1109/TSE.1976.233837
- [16] Paul Oman and Jack Hagemester. 1992. Metrics for assessing a software system’s maintainability. In *Conference on Software Maintenance*. IEEE, 337–344.
- [17] Dewayne E. Perry and Alexander L. Wolf. 1992. Foundations for the study of software architecture. 17, 4 (1992), 40–52. doi:10.1145/141874.141884
- [18] Yonnel Chen Kuang Piao, Jean Carlos Paul, L. Da Silva, Arghavan Moradi Dakhel, Mohammad Hamdaqa, and Foutse Khomh. 2025. Refactoring with LLMs: Bridging Human Expertise and Machine Understanding. In *arXiv preprint arXiv:2510.03914*. arXiv:2510.03914
- [19] Zahed Rahmati and Mohammad Tanhaei. 2021. Ensuring software maintainability at software architecture level using architectural patterns. 2, 1 (2021), 81–102. doi:10.22060/ajmc.2021.19232.1044
- [20] Seyyed Ehsan Shokri and Raffi Khatchadourian. 2024. IPSynth: Interprocedural Program Synthesis for Software Architecture Recovery and Architectural Tactics Detection. In *arXiv preprint arXiv:2403.10836*. arXiv:2403.10836
- [21] Yisen Xu, Feng Lin, Jinqiu Yang, Tse-Hsun Chen, and Nikolaos Tsantalos. 2025. MANTRA: Enhancing Automated Method-Level Refactoring with Contextual RAG and Multi-Agent LLM Collaboration. In *arXiv preprint arXiv:2503.14340*. arXiv:2503.14340